

بسمه تعالی

پروژه درس برنامه نویسی پیشرفته

دانشکده فنی انقلاب اسلامی

دانشگاه ملی مهارت

استاد درس: دکتر اردستانی

نام و نام خانوادگی دانشجو: امیر پارسا علیزاده

شماره دانشجویی: 02220040709015

نکته: در تمام این پروژه ها a آخرین رقم غیر صفر شماره دانشجویی شماست.

گزارش تحلیل کد: شبیه‌ساز چراغ راهنمایی هوشمند

۱. معرفی کلی پروژه

این برنامه یک شبیه‌ساز تعاملی چهارراه با چراغ راهنمایی چرخشی (Round-Robin Traffic Light) است که رفتار صف خودروها، زمان‌بندی چراغ‌ها، و حرکت آن‌ها هنگام عبور از تقاطع را مدل‌سازی می‌کند.

۲. دسته‌بندی بخش‌های کد

بخش	کلاس/محدوده	نقش
ثابت‌های پیکربندی سراسری	GREEN_LIGHT_SECONDS, YELLOW_LIGHT_SECONDS, ...	تنظیمات پیش‌فرض شبیه‌سازی
موجودیت‌های پایه	Car, TrafficLight	واحد‌های سیستم
مدیریت صف هر جهت	Lane	نگهداری صف خودرو و چراغ مربوطه
مدل کلی تقاطع	Intersection	نگهداری چهار Lane
منطق زمان‌بندی و تصمیم‌گیری	Controller	ماشین‌های حالت چراغ‌ها + رهاسازی خودرو
رابط کاربری و رندر	SimulatorGUI	رسم، اسلایدر تنظیمات، حلقه‌ی اجرا

۳. تحلیل موجودیت‌های پایه

۳.۱. کلاس Car

```
class Car:
    _id_counter = 1
    def __init__(self):
        self.id = Car._id_counter
        Car._id_counter += 1
```

مشابه کلاس Passenger در کد اول، از یک شمارنده‌ی سطح کلاس (Class-level Counter) برای تولید شناسه‌ی یکتا استفاده شده است.

۳.۲. کلاس TrafficLight

```
COLORS = {"red": "#e74c3c", "yellow": "#f1c40f", "green": "#2ecc71"}
```

```
@property
def is_green(self):
    return self.state == "green"
```

رنگ‌ها در یک دیکشنری سطح کلاس (COLORS) نگه داشته شده، نه در منطق رسم؛ این یعنی مسئولیت "چه رنگی نمایش داده شود" به خود کلاس منطقی واگذار شده، نه به لایه‌ی گرافیکی — نمونه‌ای صحیح از جداسازی داده از نمایش (Separation of Data and Presentation)، هرچند رنگ به‌ظاهر یک ویژگی بصری است.

۴. تحلیل کلاس Lane

```
class Lane:
    def __init__(self, direction):
        self.queue = deque()
        self.light = TrafficLight()

    def release_car(self):
        if self.light.is_green and self.queue:
            return self.queue.popleft()
        return None
```

استفاده از collections.deque به‌جای لیست معمولی برای پیاده‌سازی صف (Queue).

برخلاف list.pop(0) که پیچیدگی زمانی $O(n)$ دارد (چون تمام عناصر باید یک واحد جابه‌جا شوند)، متد deque.popleft() عملیاتی با پیچیدگی $O(1)$ است. با توجه به این که release_car مکرراً در حلقه‌ی اصلی برنامه فراخوانی می‌شود

همچنین متد release_car دو شرط را همزمان بررسی می‌کند (چراغ سبز بودن و غیرخالی بودن صف) طراحی این متد باعث می‌شود منطق "آیا خودرویی مجاز به عبور است؟" به‌طور کامل در خود Lane کپسوله شود و کلاس Controller فقط نتیجه را دریافت کند، بدون آنکه از جزئیات داخلی صف باخبر باشد.

پیاده‌سازی __len__ امکان استفاده از تابع استاندارد len(lane) را فراهم می‌کند — استفاده از Dunder/Magic Methods برای طبیعی‌تر کردن رفتار شیء مطابق قراردادهای پایتون (Pythonic Interface).

۵. تحلیل کلاس — Controller مهم‌ترین بخش کد

۵.۱. ماشین حالت دومرحله‌ای برای چرخش جهت

```
self.order = list(Intersection.DIRECTIONS)
self.current_index = 0
self.phase = "green"
```

کنترل‌کننده از دو متغیر حالت مستقل استفاده می‌کند `current_index` (کدام جهت فعال است) و `phase` (چراغ آن جهت در چه مرحله‌ای است: سبز یا زرد). این طراحی یک **FSM تودرتو (Nested State Machine)** است — یک حالت بیرونی (نوبت کدام جهت) و یک حالت درونی (فاز چراغ آن نوبت).

۵.۲. متد `tick` — قلب هماهنگی زمانی

```
def tick(self, dt: float):
    self.phase_timer += dt
    self.release_timer += dt
    ...
    if self.release_timer >= self.release_interval:
        self.release_timer = 0.0
    ...
    if self.phase == "green" and self.phase_timer >= self.green_seconds:
        ...
```

به‌جای استفاده از چند تایمر مجزا (مثلاً چند فراخوانی `after` هم‌زمان)، تمام منطق زمان‌بندی در یک متد `tick` واحد جمع شده که در هر فریم با مقدار `dt` (delta time) فراخوانی می‌شود. این الگو، معروف به **Accumulator Pattern** یا **Delta-time-based Timer**، یک روش توسعه‌ی شبیه‌سازی است؛ زیرا:

۱. دو رویداد کاملاً مستقل (رهاسازی خودرو هر ۰.۷ ثانیه، و تغییر فاز چراغ هر ۴ یا ۱.۵ ثانیه) بدون تداخل و بدون نیاز به چند `Thread` یا چند `Callback` جداگانه مدیریت می‌شوند.
۲. تغییر پویای مقادیر `green_seconds` از طریق اسلایدر (بخش ۶) بدون نیاز به لغو و بازتعریف تایمرها به‌سادگی اعمال می‌شود — چیزی که در الگوی مبتنی بر `after()` با تأخیر ثابت (مثل دو کد قبلی) دشوارتر بود.

۵.۳. هم‌پوشانی شرط‌ها با `if/elif`

```
if self.phase == "green" and self.phase_timer >= self.green_seconds:
    ...
elif self.phase == "yellow" and self.phase_timer >= self.yellow_seconds:
    ...
```

استفاده از `elif` به‌جای دو `if` مستقل، تضمین می‌کند که در یک فراخوانی از `tick`، گذار حالت حداکثر یک بار رخ دهد (از سبز به زرد، یا از زرد به سبز بعدی) — نه هر دو در یک فراخوانی. این جزئیات، از پرسش ناخواسته‌ی حالت (State Skipping) در گام‌های زمانی نامنظم جلوگیری می‌کند.

۵.۴. `last_released_direction` به عنوان رویداد لحظه‌ای

```
self.last_released_direction = None
...
if released_car:
    self.last_released_direction = direction
```

این متغیر یک الگوی رویداد یک‌باره (**One-shot Event Flag**) است: در ابتدای هر `tick` بازنشانی می‌شود و فقط در صورت وقوع رهاسازی در همان فریم مقداردهی می‌شود. این تکنیک، جایگزین ساده‌ای برای الگوی Observer/Event Emitter کامل است و به GUI اجازه می‌دهد بدون وابستگی مستقیم به جزئیات صف، از وقوع رهاسازی مطلع شود و انیمیشن عبور خودرو را آغاز کند — نمونه‌ای از ارتباط سبک بین لایه‌ی منطق و لایه‌ی نمایش.

۶. تحلیل کلاس `SimulatorGUI`

۶.۱. اسلایدرهای تعاملی برای تغییر پارامتر در حین اجرا

```
def _on_green_change(self, value):
    self.controller.green_seconds = float(value)
```

این بخش امکان تنظیم پارامترهای شبیه‌سازی در زمان اجرا (**Runtime Configuration**) را بدون توقف یا ریست برنامه فراهم می‌کند. از نظر معماری نرم‌افزار، این طراحی نشان می‌دهد که `Controller` پارامترهایش (`green_seconds`) را به صورت `Public` در معرض تغییر قرار داده و GUI مستقیماً آن را به‌روزرسانی می‌کند. راه‌حلی ساده اما مؤثر برای این مقیاس از برنامه.

۶.۲. مدل‌سازی موقعیت با نسبت پیشرفت (`progress`)

```
self.moving_cars.append({"direction": ..., "progress": 0.0})
step = dt / CROSS_SECONDS
for car in self.moving_cars:
    car["progress"] += step
```

هر خودروی در حال عبور با یک عدد بین ۰ و ۱ (`progress`) نمایش داده می‌شود که میزان طی شدن مسیر عبور از تقاطع را نشان می‌دهد.

$$x = x_1 + (x_2 - x_1) * \text{progress}$$

مزیت این روش نسبت به محاسبه‌ی مستقیم مختصات در هر فریم، جداسازی زمان‌بندی حرکت (`progress`) از مسیر هندسی حرکت (`_crossing_path`) است — تغییر شکل مسیر (مثلاً پیچ به‌جای خط مستقیم) بدون تغییر منطق زمان‌بندی امکان‌پذیر می‌شود.

۶.۳. فیلتر کردن خودروهای عبورکرده

```
self.moving_cars = [c for c in self.moving_cars if c["progress"] < 1.0]
```

استفاده از **List Comprehension** برای حذف خودروهایی که عبورشان کامل شده ($\text{progress} \geq 1.0$)، یک روش پایتونیک استاندارد برای فیلتر کردن لیست در حین پیمایش.

۶.۴. تابع کمکی مسیر بر اساس جهت

```
def _crossing_path(self, direction):  
    ...  
    if direction == "N": ...  
    if direction == "S": ...
```

این گونه تفکیک منطق هندسی بر اساس جهت، اگرچه از نظر DRY کاملاً بهینه نیست (تکرار الگوی مشابه برای چهار جهت)، اما خوانایی بالایی دارد و برای تعداد ثابت و کوچک حالات (فقط ۴ جهت)، انتخابی معقول است.

۷. جدول جمع‌بندی مفاهیم اصلی

بخش	مفهوم کلیدی
<code>Lane.queue = deque()</code>	انتخاب ساختار داده بر اساس پیچیدگی زمانی عملیات ($O(1)$ vs $O(n)$)
<code>Controller.tick()</code>	الگوی <code>Accumulator / Delta-time</code> برای چند تایمر مستقل
<code>if/elif</code> گذار فاز	جلوگیری از پرسش هم‌زمان چند حالت
<code>last_released_direction</code>	الگوی رویداد یک‌باره (<code>One-shot Event Flag</code>)
<code>moving_cars</code> در <code>progress</code>	درون‌یابی خطی پارامتریک، جداسازی زمان از مسیر
اسلایدرهای تنظیم زنده	پیکربندی در زمان اجرا (<code>Runtime Configuration</code>)
<code>TrafficLight.COLORS</code>	کپسوله‌سازی داده‌ی نمایشی درون منطق دامنه